

5. Phase 2 ADS XBRL GL Syntax Binding: Environment, Conversion Operations, and Function-Level Processing

This document isolates **src/syntax_binding_ads_xbri_gl.py**, which generates ADS XBRL GL instances from Phase 1 Structured CSV.

The process uses **semantic_path** to select Structured CSV values and row scopes, then applies XBRL GL **xpath**, selectors, **element**, **unitRef**, and **decimals** rules. It is therefore documented separately from the flat-file **semantic_binding.py** converter.

Processing Flow

```
Structured CSV + ADS XBRL GL syntax binding
→ load bindings and currency minor units
→ validate hierarchical row scopes
→ select document/VAT/line rows
→ create tuple containers from XPath/selectors
→ write items, contexts, units, and documentInfo
→ reorder XBRL GL elements and write XBRL
```

Environment and Binding Definitions

Binding Files and Review Workbook

Six syntax-binding CSVs under **specs/bindings/syntax/** define:

- Invoices Received;
- Invoices Generated;
- Invoices Received Lines;
- Invoices Generated Lines;
- Supplier Listing;
- Customer Master.

The review workbook is:

```
specs/bindings/ADS_XBRL_GL_Bindings.xlsx
```

Operation Example

Tutorial Command

```
& $python .\src\tutorial\03_generate_ads_xbrl_gl.py
```

Output directory:

```
out/tutorial/xbrl-gl/
```

The wrapper uses:

```
src/syntax_binding_ads_xbrl_gl.py  
specs/bindings/syntax/ADS_Invoices_Received_XBRL_GL_Binding.csv  
specs/lhm/EN16931_CIUS_Invoice_LHM.csv  
specs/Currency.csv
```

The generated instance contains XBRL contexts and units plus the XBRL GL tuple hierarchy required by the target view.

Conversion Contract

Scope and Selector Requirements

ADS XBRL GL instances are generated by **src/syntax_binding_ads_xbrl_gl.py**. The same semantic paths and class scopes used by ADS PSV feed six XBRL GL target views. The generator creates contexts, units, document information, selectors, and target facts according to the selected binding.

Seller identity uses the identifier-reference selector where **identifierType=V**. Seller postal-address facts belong below the matching **identifierAddress** container. Binding-table join generation and XBRL construction must preserve this selector ancestry; otherwise the Seller list appears to omit its address even when Phase 1 contains it.

Core Functions

- **load_bindings** reads and normalizes ADS XBRL GL binding rows.
- **validate_hierarchical_row_scopes** enforces the same parent/repeated-child separation as Phase 1.
- **rows_for_binding** selects source rows for each semantic fact.
- **unit_ref_from_rule** and **decimals_for_item** derive XBRL units and decimal accuracy.
- **path_parts**, **parse_path_step**, and **selector_matches** interpret target XBRL GL paths and selector predicates.
- **container_for_path** and **ensure_child** create the exact selected target hierarchy, including identifier-reference address containers.
- **append_item** creates facts while suppressing empty numeric values.

- **build_instance** creates contexts, units, document information, and bound facts for one target view.
- **reorder_tree** serializes children in stable target order.

Function-Level Implementation

Phase 2 Syntax Binding: Structured CSV To ADS XBRL GL

Script:

```
src/syntax_binding_ads_xbri_gl.py
```

Main functions:

```
load_bindings(...)
validate_hierarchical_row_scopes(...)
build_instance(...)
convert_file(...)
```

Input:

- Phase 1 Structured CSV.
- ADS XBRL GL syntax binding CSV.
- Currency minor unit table, **specs/Currency.csv**.

Output:

- One of six ADS XBRL GL tuple views: **Invoices_Received.xbri**, **Invoices_Generated.xbri**, **Invoices_Received_Lines.xbri**, **Invoices_Generated_Lines.xbri**, **Supplier_Listing.xbri**, or **Customer_Master.xbri**.

Binding Interpretation

ADS XBRL GL bindings use **semantic_path** and **structured_csv_column** to select the source Structured CSV value. They use **element** and **xpath** to identify the target XBRL GL item and tuple location.

Before building the XBRL GL instance, **validate_hierarchical_row_scopes(...)** derives each fact's owning dimension from binding **type=C**, **multiplicity**, and **semantic_path** rows. A non-repeating child belongs to its nearest repeating ancestor row. A repeating child owns a separate row from its first occurrence. The converter rejects input that populates parent facts and repeating-child facts on the same child row.

Example binding dictionary:

```
{
  "source_column": "InvoiceLineIdentifier",
  "semantic_path": "$.invoice.invoiceLine.invoiceLineIdentifier",
  "element": "gl-cor:documentNumber",
  "xpath": "/xbrli:xbrl/gl-cor:accountingEntries/...",
  "value_source": "structured_csv_column",
  "unit_ref_rule": "",
  "decimals": ""
}
```

Unlike **syntax_binding.py**, this script keeps binding rows as dictionaries.

```
bindings: list[dict[str, str]]
rows: list[dict[str, str]]
currency_minor_units: dict[str, str]
```

ADS Dimension Ownership And Input Validation

load_bindings(binding_csv) retains the binding properties needed for both XBRL construction and Structured CSV validation:

- **type** distinguishes class (**C**) and fact (**A**) rows;
- **multiplicity** determines whether a class repeats;
- **semantic_path** defines class ancestry;
- **structured_csv_column** becomes **source_column**;
- **element** and **xpath** identify the XBRL GL target.

validate_hierarchical_row_scopes(rows, bindings) performs these steps:

1. It scans **type=C** rows. The invoice root and classes for which **multiplicity_repeats(...)** is true are registered in **class_dimensions**.
2. **dimension_name(source_column)** converts a class column such as **InvoiceLine** into **dInvoiceLine**.
3. For every **type=A** fact, it removes semantic path segments from the right until it finds the nearest registered class dimension. The result is stored in **field_dimensions**.
4. For each Structured CSV row, it finds the deepest populated **dXxx** column.
5. It rejects the row if any populated ADS-bound fact belongs to a different dimension.

This means a single child class has no independent row scope and its facts belong to the nearest repeating ancestor. A repeating child has its own row scope from occurrence 1. Parent facts and repeating-child facts cannot be populated together on that first child row.

The validation is called by **convert_file(...)** immediately after **read_csv(...)** and before **build_instance(...)**, so invalid hierarchy is not partially written to an XBRL GL instance.

ADS Row Selection

After validation, **rows_for_binding(rows, binding)** selects the sparse input rows used by each fact binding:

- VAT breakdown facts require a populated **dVatBreakdown**;
- invoice-line facts require a populated **dInvoiceLine**;
- document-level facts use the first row containing the source value.

first_non_empty(...) obtains document-wide currency, date, and entity values. The selected source row is also passed to **unit_ref_from_rule(...)**, so a monetary fact can obtain its unit from a bound Structured CSV currency column.

XBRL XML Construction

build_instance(...) creates an **ET.Element** tree rooted at **xbri:xbri**. It adds context, units, XBRL GL document metadata, and the tuple containers required by the selected binding paths. Invoice bindings create document structures; Supplier Listing and Customer Master create the applicable identifier reference structures.

Path handling:

- **path_parts(xpath)** splits the target XPath into steps.
- **parse_path_step(step)** separates an element name from selector conditions.
- **container_for_path(root, xpath, currency, monetary_decimals)** walks the tuple path.
- **ensure_child(...)** finds or creates tuple containers.
- **selector_matches(...)** checks whether an existing element satisfies selector child values.
- **append_item(...)** creates the final item element and writes the value.

The Supplier Listing binding selects the vendor identifier with **gl-cor:identifierReference[gl-cor:identifierType="V"]**. Seller postal address facts use target paths below that selected identifier reference and create **gl-bus:identifierAddress** with **identifierStreet**, **identifierAddressStreet2**, **identifierCity**, **identifierStateOrProvince**, **identifierCountry**, and **identifierZipOrPostalCode** children. The selector identifies the vendor tuple; it does not create address children unless corresponding fact rows are present in the binding table.

The Seller address must not be written as **gl-bus:organizationAddress**. That tuple describes the reporting entity under **entityInformation**. A supplier represented by the vendor identifier reference uses **gl-bus:identifierAddress** below the same **identifierType=V** tuple.

Rows are selected by **rows_for_binding(rows, binding)**. A binding for invoice lines selects rows where the relevant source column and line dimension are present. A document-level binding selects the document row.

Numeric facts use:

- **unit_ref_from_rule(...)** for **unitRef**.
- **decimals_for_item(...)** for **decimals**.

- **load_currency_minor_units(...)** to get currency minor units from **specs/Currency.csv**.
- **add_missing_units(...)** to ensure every referenced unit is declared.

Construction then finishes as follows:

1. **make_xbri_root(...)** creates **xbri:xbri** and **schemaRef**.
2. **add_context_and_units(...)** adds the default context and initial units.
3. **add_document_info(...)** writes required XBRL GL document metadata.
4. **build_instance(...)** writes VAT occurrences and other bound facts.
5. **reorder_tree(...)** applies the supported XBRL GL child order, including **identifierAddress** under **identifierReference** and the schema order of the address facts.
6. **add_missing_units(...)** declares any unit referenced by generated facts.
7. **convert_file(...)** indents and writes the XML document.
8. **main(...)** resolves one file or directory with **input_files(...)** and derives output names with **binding_target_stem(...)** and **schema_href_for_output(...)**.

Regression Tests

Regression Test Overview

Run all target-specific scripts named **test_ads_*_xbri_gl.py**, followed by:

```
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

Complete Test Commands

These tests regenerate Phase 1 Structured CSV first, then use ADS XBRL GL syntax binding CSV files under **specs/bindings/syntax/**.

```
& $python .\tests\test_ads_invoices_received_xbri_gl.py
& $python .\tests\test_ads_invoices_generated_xbri_gl.py
& $python .\tests\test_ads_invoices_received_lines_xbri_gl.py
& $python .\tests\test_ads_invoices_generated_lines_xbri_gl.py
& $python .\tests\test_ads_supplier_listing_xbri_gl.py
& $python .\tests\test_ads_customer_master_xbri_gl.py
```

Expected output:

```
out/phase2/ADS_XBRL_GL/<structured-csv-stem>/<target-view>.xbri
```

Target view files:

```
Invoices_Received.xbrl  
Invoices_Generated.xbrl  
Invoices_Received_Lines.xbrl  
Invoices_Generated_Lines.xbrl  
Supplier_Listing.xbrl  
Customer_Master.xbrl
```

test_ads_supplier_listing_xbrl_gl.py verifies that Seller name and identifier are written to the **identifierType=V** identifier reference and that Seller Street, City, Country, and Postal Code are written below its **gl-bus:identifierAddress** tuple.